Práctica 3.6 — Infraestructura como código (AWS)

Objetivo: crear con Terraform todo el entorno de Nebula Arena, revisarlo como código y destruirlo al terminar sin dejar recursos que sigan cobrando.

Se usó la **variante AWS** del material. La carpeta es `infra-aws/`; la versión de Google Cloud sigue en `infra/` como alternativa.

Qué crea Terraform (un solo `terraform apply`)

Archivo	Recursos
`red.tf`	VPC 10.0.0.0/16, 2 subredes públicas, internet gateway, regla UDP/TCP 7000–8000 para los servidores
`cluster.tf`	EKS 1.36, grupo de nodos `game-pool` (m7i-flex.large, 2–6), complementos `eks-pod-identity-agent` y `metrics-server`, permisos del Cluster Autoscaler
`complementos.tf`	Helm: Agones 1.62, Cluster Autoscaler y kube-prometheus-stack (Prometheus + Grafana)
`registro.tf`	ECR `nebula-server` y `matchmaker` (etiquetas inmutables, escaneo al subir, se guardan 10 imágenes)
`tabla_lideres.tf`	DynamoDB, Lambda, API Gateway y logs (práctica 3.5)
`github.tf`	Usuario IAM del pipeline con permisos mínimos y su acceso al clúster (práctica 3.2)
`presupuesto.tf`	Alarma de presupuesto mensual (ver abajo)
`providers.tf`	Proveedores, etiquetas `proyecto=nebula-arena` en todo, **estado remoto en S3**

Los manifiestos del juego (flota, FleetAutoscaler, matchmaker, PodMonitor y RBAC) los aplica `desplegar-juego.ps1`, y después los actualiza el pipeline.

Conceptos del material, aplicados

Concepto	Dónde se ve
Declarativo	Los `.tf` describen el estado final ("6 nodos como máximo"), no los pasos.
Estado remoto con bloqueo	`s3://nebula-tfstate-<cuenta>/nebula/aws.tfstate`: bucket privado, versionado y cifrado. El bloqueo usa `use_lockfile` (nativo de S3 desde Terraform 1.10), así que no hace falta la tabla de DynamoDB del material. El estado local se migró con `terraform init -migrate-state`.
Drift	`terraform plan` contra la infraestructura viva, después de la prueba de carga: revisó 45 recursos y solo propuso agregar el presupuesto nuevo (`01-plan-sin-drift.txt`). El autoescalador cambia `desired_size` de los nodos; `ignore_changes` evita que Terraform lo vea como drift y lo regrese.
Idempotencia	Un segundo `plan` después del `apply` respondió `No changes` (`02-idempotencia.txt`).
plan → apply	Siempre `plan -out` y `apply` del archivo del plan: se aplica exactamente lo revisado.
Secretos fuera del código	La llave de la tabla de líderes vive en SSM y no pasa por Terraform. El correo de alertas y el ID de cuenta no están en el repositorio.

Alarma de costos

El plan gratuito descuenta todo de los créditos, así que el costo neto siempre aparece en USD 0. Por eso el presupuesto `nebula-arena-mensual` mide el costo **antes** de créditos (`include_credit = false`), con un límite de USD 15 al mes. Avisa por correo al 50 % y al 80 % del gasto real, y al 100 % del pronóstico del mes. Se suma al presupuesto "Zero-Spend" de USD 1 que ya tenía la cuenta.

Costo por hora (precios oficiales, us-east-2)

Precios bajo demanda consultados con la API de precios de AWS (`aws pricing get-products`), 2026-11-09:

Recurso	Precio	En reposo (2 nodos)	En el pico (6 nodos)
Plano de control de EKS	USD 0.10/h	0.100	0.100
Nodo m7i-flex.large	USD 0.09576/h	0.192	0.575
IPv4 pública (nodos + balanceador)	USD 0.005/h c/u	≈ 0.020	≈ 0.040
Balanceador clásico del matchmaker	USD 0.025/h	0.025	0.025
Tabla de líderes (Lambda, API Gateway, DynamoDB)	por uso	≈ 0	≈ 0
Total		**≈ USD 0.34/h**	**≈ USD 0.74/h**

No incluye los discos EBS de 20 GB por nodo ni la transferencia de datos.

- **Costo por jugador** en el pico de la 3.4 (252 jugadores, 6 nodos): 0.74 / 252 ≈ **USD 0.003 por jugador-hora**.
- **Por qué hay que destruir al terminar:** el clúster en reposo cuesta ≈ USD 8 al día. Los USD 134 de crédito durarían unos 16 días con el clúster encendido sin usarse.
- **Gasto real registrado** (Cost Explorer, 1–9 nov, antes de créditos): EC2 USD 2.12, IPv4 públicas USD 1.03, otros de EC2 USD 0.18, S3 y Secrets Manager < USD 0.01. El cargo de EKS del día todavía no aparece: Cost Explorer tarda hasta 24 h.

Ciclo completo

```
terraform init "-backend-config=bucket=nebula-tfstate-<cuenta>"
terraform fmt -check; terraform validate
terraform plan -out plan.tfplan
terraform apply plan.tfplan
.\desplegar-juego.ps1 -Etiqueta <sha>
# ... pruebas de la 3.4 ...
terraform destroy
```

Los pasos completos, incluida la creación del bucket del estado, están en `infra-aws/README.md`.

Evidencias (`entrega/evidencias/3.6-terraform/`)

Archivo	Qué muestra
`01-plan-sin-drift.txt`	`plan` completo contra la infraestructura viva: sin drift, solo el presupuesto nuevo
`02-idempotencia.txt`	segundo `plan`: `No changes`
`03-cluster-funcionando.txt`	EKS `ACTIVE`, nodos, Agones, autoescalador, monitoreo, flota, HPA y matchmaker
`04-destroy.txt`	`terraform destroy` de los 46 recursos y comprobación de que no quedó nada del proyecto cobrando

Destroy sin recursos huérfanos

1. **Antes del destroy** se borraron el balanceador del matchmaker y la flota. Los crea Kubernetes, no Terraform; sin este paso, el balanceador habría quedado huérfano y habría bloqueado el borrado de la red.
2. `terraform plan -destroy -out destroy.tfplan` → **46 to destroy**, y `apply` de ese plan.
3. **Dos fallas reales encontradas y corregidas:** - **Orden de borrado:** Terraform borró la ruta a internet antes que Agones. El desinstalador de Agones (un *hook* de Helm) ya no pudo descargar su imagen y el destroy se atoró. Se quitó el job atorado y se corrigió `cluster.tf`: el grupo de nodos ahora depende de la ruta, así que la ruta se borra al final. - **Usuario de GitHub:** `force_destroy` intenta `iam:DeleteLoginProfile`, y la política de la organización del plan gratuito lo niega. Se quitó `force_destroy`. Su llave, creada fuera de Terraform, se borra con la CLI antes del destroy (ver `github.tf`).
4. **Resultado:** 46 de 46 recursos destruidos y `terraform state list` vacío. La comprobación en AWS no encontró clústeres, balanceadores, IPs, VPC, ECR, Lambda, DynamoDB, API Gateway ni usuarios o roles del proyecto. Solo quedan, a propósito y con costo prácticamente cero, el bucket del estado y la llave en SSM.